

This function transforms raw byte counts into human-readable formats (B, KB, MB, GB, TB). This is essential for the `-h` option, making large file sizes easier to understand at a glance.

```
get_file_info(path, filename)
def get_file_info(path, filename):
    """Get file information"""
```

The information gatherer collects all metadata for a single file or directory including size, modification time, permissions, and Windows attributes. It returns a structured dictionary for easy processing.

```
list_directory(path, args)
def list_directory(path, args):
    """List directory contents"""
```

This is the main orchestrator that reads directory contents, filters hidden files (unless `-a` is specified), applies sorting based on flags (`-t` for time, `-S` for size), and delegates to the appropriate display function.

display_long_format(file_infos, args) and display_short_format(file_infos, args): These functions handle output formatting. Long format (`-l`) shows detailed information in columns, while short format displays files in a multi-column layout optimised for terminal width.

Supported options are:

- `-a, --all`: Show hidden files (including those starting with `.`)
- `-l, --long`: Long listing format with permissions, size, and date
- `-h, --human-readable`: Display sizes in KB, MB, GB instead of bytes
- `-t, --time`: Sort by modification time
- `-S, --size`: Sort by file size
- `-r, --reverse`: Reverse sort order
- `-F, --classify`: Append indicators (`/` for directories, `*` for executables)
- `-1, --one`: One file per line
- `--attrs`: Show Windows file attributes (H, S, R, A)

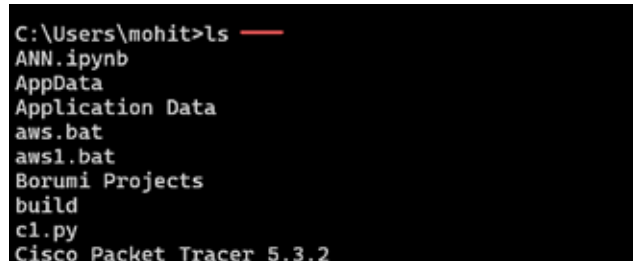
The usage of the `ls` command is shown in Figures 1 and 2.

The *grep* command: Pattern matching made simple

The *grep* command searches for text patterns in files or piped input, bringing UNIX text-processing power to Windows.

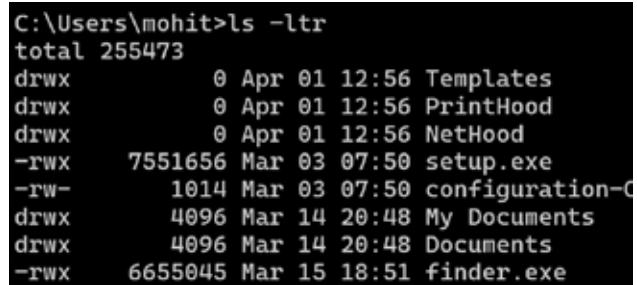
The core functions are:

```
grep(pattern, lines, case_sensitive, invert, count_only, show_line_num)
python
def grep(pattern, lines, case_sensitive=True, invert=False,
        count_only=False, show_line_num=False):
```



```
C:\Users\mohit>ls
ANN.ipynb
AppData
Application Data
aws.bat
aws1.bat
Borumi Projects
build
cl.py
Cisco Packet Tracer 5.3.2
```

Figure 1: Use of `ls` command



```
C:\Users\mohit>ls -ltr
total 255473
drwx      0 Apr 01 12:56 Templates
drwx      0 Apr 01 12:56 PrintHood
drwx      0 Apr 01 12:56 NetHood
-rwx    7551656 Mar 03 07:50 setup.exe
-rw-    1014 Mar 03 07:50 configuration-C
drwx    4096 Mar 14 20:48 My Documents
drwx    4096 Mar 14 20:48 Documents
-rwx    6655045 Mar 15 18:51 finder.exe
```

Figure 2: File listing according to date

```
"""Search for pattern in lines."""
```

The heart of the tool iterates through input lines, applies regex pattern matching with optional flags, and outputs matching results. The *invert* flag enables negative matching (showing lines that DON'T match), while *count_only* returns just the number of matches.

```
main()
def main():
    """Main grep function."""
```

This function handles command-line argument parsing, determines input source (file or stdin), and invokes the *grep* function with appropriate parameters. It intelligently detects whether the input is piped or from a file.

Supported options are:

- `-i`: Case-insensitive search
- `-v`: Invert match (show non-matching lines)
- `-c`: Count matching lines only
- `-n`: Show line numbers

Usage examples for *grep* are shown in Figure 3.

Deployment: From Python to system command

Step 1: Install PyInstaller:

```
pip install pyinstaller
```

Step 2: Create executables:

```
pyinstaller --onefile ls.py
pyinstaller --onefile grep.py
```